

Lab 8: Introducing Hyperparameter Tuning

Objectives:

- To gain hands-on experience tuning parameters
- To implement concepts related to Hyperparameter Tuning

Lab

Load Necessary Libraries

```
In [73]: import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV, RandomizedSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

Load and Prepare Data

We're going to use iris dataset.

```
In [74]: # Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split into training and testing sets (80-20 split)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Let's use decision tree again! (Don't forget it yet)

```
In [75]: # Initialize a Decision Tree model
dt = DecisionTreeClassifier(random_state=42)
```

Parameter Tuning Using GridSearchCV

```
In [76]: # Define parameter grid for tuning
param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [3, 5, 10, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Apply GridSearchCV
grid_search = GridSearchCV(dt, param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train, y_train)

# Get best parameters
print("Best Parameters from GridSearchCV:", grid_search.best_params_)
```

Best Parameters from GridSearchCV: {'criterion': 'entropy', 'max_depth': 5, 'min_samples_leaf': 4, 'min_samples_split': 2}

Parameter Tuning Using RandomizedSearchCV

```
In [77]: from scipy.stats import randint

# Define parameter distributions for tuning
param_dist = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [3, 5, 10, None],
    'min_samples_split': randint(2, 20),
```

```

        'min_samples_leaf': randint(1, 10)
    }

    # Apply RandomizedSearchCV
    random_search = RandomizedSearchCV(dt, param_dist, n_iter=10, cv=5, scoring='accuracy', n_jobs=-1, random_state=42)
    random_search.fit(X_train, y_train)

    # Get best parameters
    print("Best Parameters from RandomizedSearchCV:", random_search.best_params_)

```

Best Parameters from RandomizedSearchCV: {'criterion': 'entropy', 'max_depth': None, 'min_samples_leaf': 3, 'min_samples_split': 3}

Evaluate the Best Model

```

In [78]: # Train a Decision Tree using the best parameters from GridSearchCV
best_dt = grid_search.best_estimator_
y_pred = best_dt.predict(X_test)

# Compute accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Test Accuracy with Best GridSearchCV Model: {accuracy:.4f}")

```

Test Accuracy with Best GridSearchCV Model: 1.0000

Tasks:

Use wine dataset :)

```

In [79]: from sklearn.datasets import load_wine

# Load dataset
wine = load_wine()
X = wine.data
y = wine.target

```

Task 1: Train-Test Split

- Split the Wine dataset into training and testing sets using an 80-20 split.
- What is the role of the train-test split in evaluating a machine learning model?

```

In [80]: # Write your code here
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```

Train-test split role is to split data into two part which is train data for training model and test data (Validation) for Validate model accuracy and performance to improve in next epoch.

Task 2: Cross-Validation

- Implement cross-validation using GridSearchCV and RandomizedSearchCV.
- How does cross-validation help in providing a more reliable estimate of the model's performance?
- Discuss how cross-validation improves the results and prevents overfitting compared to a simple train-test split.

```

In [81]: from sklearn.metrics import classification_report

```

```

In [82]: temp = DecisionTreeClassifier(random_state=42)
temp.fit(X_train, y_train)

```

```

Out[82]: ▼ DecisionTreeClassifier ⓘ ?
          ► Parameters

```

```

In [83]: y_pred = temp.predict(X_test)
print(classification_report(y_test, y_pred))

```

	precision	recall	f1-score	support
0	0.93	0.93	0.93	14
1	0.93	1.00	0.97	14
2	1.00	0.88	0.93	8
accuracy			0.94	36
macro avg	0.95	0.93	0.94	36
weighted avg	0.95	0.94	0.94	36

```
In [84]: # Write your code here
param_dist = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [3, 5, 10, None],
    'min_samples_split': randint(2, 20),
    'min_samples_leaf': randint(1, 10)
}

random_search = RandomizedSearchCV(dt, param_dist, n_iter=10, cv=5, scoring='accuracy', n_jobs=-1, random_state=42)
random_search.fit(X_train, y_train)

print("Best Parameters from RandomizedSearchCV:", random_search.best_params_)
```

Best Parameters from RandomizedSearchCV: {'criterion': 'entropy', 'max_depth': None, 'min_samples_leaf': 5, 'min_samples_split': 2}

```
In [85]: y_pred = random_search.predict(X_test)

print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.93	1.00	0.97	14
1	0.93	1.00	0.97	14
2	1.00	0.75	0.86	8
accuracy			0.94	36
macro avg	0.96	0.92	0.93	36
weighted avg	0.95	0.94	0.94	36

```
In [86]: param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [3, 5, 10, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Apply GridSearchCV
grid_search = GridSearchCV(dt, param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train, y_train)

# Get best parameters
print("Best Parameters from GridSearchCV:", grid_search.best_params_)
```

Best Parameters from GridSearchCV: {'criterion': 'gini', 'max_depth': 3, 'min_samples_leaf': 1, 'min_samples_split': 2}

```
In [87]: y_pred = grid_search.predict(X_test)

print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	1.00	0.93	0.96	14
1	0.88	1.00	0.93	14
2	1.00	0.88	0.93	8
accuracy			0.94	36
macro avg	0.96	0.93	0.94	36
weighted avg	0.95	0.94	0.94	36

If the test set contain easy or outlier cases, the score will be biased. Cross Validation generalized this by averaging scores across multiple folds and make the model not overfitting.

Task 3: Hyperparameter Tuning

- Use GridSearchCV to find the best hyperparameters by exhaustively searching through the parameter grid.
- Use RandomizedSearchCV to sample a fixed number of combinations of hyperparameters.
- Compare the results from both methods in terms of accuracy and computation time.
- Discuss the trade-offs between GridSearchCV and RandomizedSearchCV.

```
In [88]: # Write your code here
import time
```

```
In [89]: param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [3, 5, 10, 20, None],
    'min_samples_split': [2, 5, 10, 15],
    'min_samples_leaf': [1, 2, 5]
}
```

```
In [90]: start_grid = time.time()
grid_search = GridSearchCV(dt, param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train, y_train)
end_grid = time.time()
print(f"GridSearch Score: {grid_search.best_score_}")
print(f"GridSearch Time: {end_grid - start_grid:.4f}s")
print("Best Parameters from GridSearchCV:", grid_search.best_params_)
```

GridSearch Score: 0.9224137931034484

GridSearch Time: 0.2170s

Best Parameters from GridSearchCV: {'criterion': 'gini', 'max_depth': 3, 'min_samples_leaf': 1, 'min_samples_split': 2}

```
In [91]: start_random = time.time()
random_search = RandomizedSearchCV(dt, param_grid, n_iter=10, cv=5, n_jobs=-1, scoring='accuracy', random_state=42)
random_search.fit(X_train, y_train)
end_random = time.time()
print(f"RandomizedSearchCV Score: {random_search.best_score_}")
print(f"RandomSearch Time: {end_random - start_random:.4f}s")
print("Best Parameters from RandomizedSearchCV:", random_search.best_params_)
```

RandomizedSearchCV Score: 0.9224137931034484

RandomSearch Time: 0.0378s

Best Parameters from RandomizedSearchCV: {'min_samples_split': 10, 'min_samples_leaf': 1, 'max_depth': 10, 'criterion': 'gini'}

Random search random between parameter grid in only n iteration while grid_search choose the best parameters on every combination.

Task 4: Model Evaluation

- Using the best model from GridSearchCV or RandomizedSearchCV, make predictions on the test set and calculate the accuracy.
- Compare the performance of the tuned model with a baseline Decision Tree model (without hyperparameter tuning).
- How does hyperparameter tuning affect the accuracy of the Decision Tree model?

```
In [92]: # Write your code here
baseline_model = DecisionTreeClassifier(random_state=42)
baseline_model.fit(X_train, y_train)
y_pred_baseline = baseline_model.predict(X_test)
baseline_accuracy = accuracy_score(y_test, y_pred_baseline)
print(f"Baseline Accuracy: {baseline_accuracy:.4f}")
```

Baseline Accuracy: 0.9444

```
In [93]: best_model = grid_search.best_estimator_
y_pred_tuned = best_model.predict(X_test)
tuned_accuracy = accuracy_score(y_test, y_pred_tuned)
print(f"Tuned Accuracy: {tuned_accuracy:.4f}")
```

Tuned Accuracy: 0.9444

```
In [95]: baseline_pred = baseline_model.predict(X_test)
print(classification_report(y_test, baseline_pred))
```

	precision	recall	f1-score	support
0	0.93	0.93	0.93	14
1	0.93	1.00	0.97	14
2	1.00	0.88	0.93	8
accuracy			0.94	36
macro avg	0.95	0.93	0.94	36
weighted avg	0.95	0.94	0.94	36

```
In [94]: grid_pred = random_search.predict(X_test)
print(classification_report(y_test, grid_pred))
```

	precision	recall	f1-score	support
0	1.00	0.93	0.96	14
1	0.88	1.00	0.93	14
2	1.00	0.88	0.93	8
accuracy			0.94	36
macro avg	0.96	0.93	0.94	36
weighted avg	0.95	0.94	0.94	36

In this dataset hyperparameter tuning doesn't affect the accuracy of the Decision Tree model but if we look at the misprediction case it has some different so if we use it on the larger dataset we can improve and generalize the model using hyperparameter tuning.